



Taller 2

PyGroup

1. Fundamentos Teóricos y Sintaxis

Repasa las herramientas que necesitarás para resolver el taller. Todo el ejercicio se resuelve usando únicamente tipos primitivos, operadores, condicionales, ciclos y colecciones nativas.

A. Indexación y Slicing en Listas

Permite extraer fragmentos usando `secuencia[inicio:fin:paso]`. El índice final no se incluye.

```
numeros = [10, 20, 30, 40, 50, 60]
primeros_dos = numeros[:2]      # [10, 20]
ultimos_dos = numeros[-2:]     # [50, 60]
invertida = numeros[::-1]      # [60, 50, 40, 30, 20, 10]
```

B. Comprensión de Listas y Diccionarios

Construye nuevas colecciones en una sola línea a partir de un ciclo `for` y filtros condicionales:

```
notas = [3.5, 4.8, 2.9, 4.0, 5.0]
aprobadas = [n for n in notas if n >= 3.0]
cuadrados = {n: n**2 for n in [1, 2, 3]} # {1: 1, 2: 4, 3: 9}
```

C. Lectura y Escritura de Archivos Planos (.txt)

Los archivos de texto no conservan tipos de datos (todo entra como texto). Se debe convertir (*castear*) cada valor manualmente:

```
# --- LECTURA ---
with open("origen.txt", "r", encoding="utf-8") as f:
    for linea in f:
        linea_limpia = linea.strip() # Quita espacios y el salto de linea \n
        if linea_limpia:             # Salta líneas vacías
            partes = linea_limpia.split(",") # Divide por delimitador

# --- ESCRITURA ---
with open("destino.txt", "w", encoding="utf-8") as f:
    f.write("Resultado final:\n")
    f.write("Promedio: " + str(4.2) + "\n")
```

2. Enunciado del Problema

El capítulo estudiantil consolidó las calificaciones de dos espacios formativos en un único archivo plano denominado `datos_talleres.txt`: el **Taller de Algoritmia** y el **Taller de Desarrollo Web**.

El archivo contiene dos bloques divididos por encabezados, líneas con registros duplicados idénticos y casos de estudiantes con el mismo nombre y apellido (homónimos) que se diferencian únicamente por su código estudiantil.

Debes construir un script de Python que lea el archivo en una sola pasada con un ciclo, use condicionales para detectar en qué bloque está, limpie los datos, calcule métricas de conjuntos y obtenga estadísticas descriptivas mediante operadores aritméticos y bucles.

Estructura del Archivo de Entrada (`datos_talleres.txt`)

- **Línea 0 (Encabezado 1):** `Codigo,Nombre,Apellido,Taller_Algoritmia_N1,N2`
- **Líneas 1 a $m-1$:** Registros del formato `<codigo>,<nombre>,<apellido>,<nota1>,<nota2>`.
- **Línea m (Encabezado 2):** `Codigo,Nombre,Apellido,Taller_Web_N1,N2`
- **Líneas $m+1$ a j :** Registros del segundo taller con la misma estructura.

3. Requerimientos del Sistema

Tu código debe resolver secuencialmente los siguientes pasos:

1. **Lectura con Conmutación de Contexto:** Abre el archivo `datos_talleres.txt` en modo lectura. Mediante un ciclo `for` y sentencias `if / elif`, detecta cuándo inicia cada taller para clasificar los registros en dos listas: `raw_algoritmia` y `raw_web`. Cada elemento debe ser una tupla: `(codigo, nombre, apellido, float(nota1), float(nota2))`.
2. **Limpieza y Slicing:**
 - a) Elimina las filas duplicadas exactas de cada lista sin usar bucles anidados, aprovechando las propiedades de los conjuntos (`set`) y volviendo a convertir a lista.
 - b) Usando *slicing*, extrae y muestra por consola los 3 primeros y los 3 últimos registros de la lista depurada de Algoritmia.
3. **Métricas de Participación (Sets):** Crea dos conjuntos que contengan únicamente los códigos estudiantiles de cada taller. Empleando operadores de conjuntos (`&`, `^`, `<=`), calcula:
 - a) Los códigos de quienes cursaron ambos talleres (asistencia total).
 - b) Los códigos de quienes cursaron exactamente un taller (asistencia única).
 - c) Si el grupo de Web es un subconjunto de Algoritmia (booleano).
4. **Consolidación en Diccionario Central:** Construye una variable llamada `base_de_datos` donde cada llave sea el código y su valor sea un diccionario con:
 - `"nombre_completo"`: Cadena concatenada `("Nombre Apellido")`.
 - `"talleres"`: Lista con los talleres cursados (`[.Algoritmia]`, `["Web"]` o ambos).

- "definitiva_algoritmia": Promedio de sus dos notas, o None si no participó.
- "definitiva_web": Promedio de sus dos notas, o None si no participó.

5. **Métricas Estadísticas Descriptivas (Iteraciones y Operadores):** Utilizando ciclos o comprensión de listas (sin librerías como `math` o `statistics`):

- Calcula la media ($\bar{x}$) de las notas definitivas de Algoritmia: sumatoria dividida entre la cantidad de estudiantes.
- Calcula la varianza muestral (s^2) y la desviación estándar (s) de las definitivas de Algoritmia usando operadores aritméticos:

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}, \quad s = (s^2)^{0.5}$$

- Mediante comprensión de listas, filtra los nombres de los estudiantes de Algoritmia cuya nota definitiva sea mayor a la media ($\bar{x}$).

6. **Mutabilidad y Manejo Seguro:**

- El estudiante con código "2024105" canceló semestre. Extráelo de `base_de_datos` usando el método `.pop()` y almacénalo en un diccionario llamado `backup`.
- Consulta la información del estudiante "2024999" usando `.get()`. Si no existe, muestra el mensaje `.Estudiante no encontrado`.

7. **Escritura del Reporte:** Crea un archivo nuevo llamado `reporte_final.txt` en modo 'w' y escribe allí el total de estudiantes procesados, la media y desviación calculadas, y el listado de estudiantes que superaron la media.

4. Criterios de Evaluación

La calificación del taller considerará los siguientes aspectos:

- **Selección de Estructuras (25 %):** Uso correcto de tuplas para registros inmutables, listas para secuencias ordenadas, conjuntos para operaciones algebraicas y diccionarios para consultas directas.
- **Lógica de Flujo y Archivos (25 %):** Lectura correcta en una sola pasada con detección dinámica de bloques, conversión adecuada de tipos (*casting*) y generación limpia del archivo de reporte.
- **Cálculos Estadísticos Nativos (20 %):** Implementación precisa de media, varianza muestral y desviación estándar utilizando operadores aritméticos elementales o comprensiones, sin recurrir a módulos externos.
- **Manejo Seguro y Mutabilidad (15 %):** Aplicación estricta de métodos seguros (`.get()`, `.pop()`) garantizando que el programa no falle ante registros faltantes.
- **Estilo y Restricciones (15 %):** Cumplimiento de la restricción de cero `import`, ejecución secuencial limpia y presencia del encabezado de identificación.

5. Estructura del Archivo de Entrega (`solucion.py`)

El entregable debe ser un único archivo script de Python con el siguiente formato:

- **Encabezado:** Comentarios al inicio con nombre completo, código estudiantil y fecha.
- **Sin Librerías Externas:** No se permite el uso de ningún `import` (`math`, `statistics`, `csv`, etc.).